Assignment No. 01

Title: A. Create Binary tree and perform following operations:

a. Insert
b. Display
c. Find Depth of a tree
d. Display leaf-nodes
e. Create a copy of a tree.

B. Implement binary search tree and perform following operations:

a. Insert b. Delete c. Display d. Search e. BFS (Level wise print)

Introduction:

- **Tree Data Structure**

  A Tree is a non-linear, hierarchical data structure consisting of a collection of nodes connected by edges. Unlike arrays or linked lists, which are linear (elements follow one after another), trees represent relationships like a family tree or a file directory system.
  - Recursive Nature: A tree is composed of a root node, and every node can be the root of its own subtree.
  - Properties: A tree with n nodes will always have exactly n-1 edges. There are no loops or cycles in a tree.

- **Binary Tree vs. Binary Search Tree (BST)**

  1. Binary Tree (BT)

  A Binary Tree is the simplest form of a tree where each node has at most two children (0, 1, or 2). There is no specific order in how data is arranged. If you insert "10" and then "5", "5" could go anywhere as long as it's a child of an existing node.

  2. Binary Search Tree (BST)

  A Binary Search Tree is a "sorted" binary tree. It follows a specific relative ordering:
  - Left Subtree: All values are less than the parent node.
  - Right Subtree: All values are greater than the parent node.
  - No Duplicates: Typically, BSTs do not allow duplicate values (though some variations do).

- **Tree Traversals**

  Traversal refers to the process of visiting every node in the tree exactly once. Because trees are non-linear, we have two main strategies: Depth-First Search (DFS) and Breadth-First Search (BFS).

  A. Depth-First Search (DFS)

  DFS dives deep into a branch before backtracking. There are three standard ways to do this:
  1. **In-Order (Left, Root, Right):** Gives you the values in ascending sorted order.
  2. **Pre-Order (Root, Left, Right):** Best for creating a copy of the tree or evaluating prefix expressions.
  3. **Post-Order (Left, Right, Root):** Best for deleting a tree (bottom-up) or calculating the size of a directory.

  B. Breadth-First Search (BFS) / Level-Order

  BFS visits nodes level-by-level, starting from the root (Level 0), then all nodes at Level 1, and so on.

- **Mechanism:** It uses a Queue (First-In-First-Out). You "enqueue" the root, then repeatedly "dequeue" a node, visit it, and "enqueue" its left and right children.
- **Usage:** Finding the shortest path between two nodes or printing the tree exactly as it looks visually.

Algorithms:
A. Binary Tree -
1. Binary Tree Insertion
Step 1: Start
Step 2: Create new node using CREATE_NODE(data)
Step 3: If root == NULL
Step 4: Return new node
Step 5: Create empty queue
Step 6: Insert root into queue
Step 7: While queue is not empty
Step 8: Remove front node → temp
Step 9: If temp.left == NULL
Step 10: Set temp.left = newNode
Step 11: Return root
Step 12: Else insert temp.left into queue
Step 13: If temp.right == NULL
Step 14: Set temp.right = newNode
Step 15: Return root
Step 16: Else insert temp.right into queue
Step 17: End While
Step 18: Stop

2. Binary Tree Traversal
Step 1: Start
Step 2: If root == NULL, return
Step 3: Call INORDER(root.left)
Step 4: Print root.data
Step 5: Call INORDER(root.right)
Step 6: Stop

3. Display Leaf Nodes
Step 1: Start
Step 2: If root == NULL, return
Step 3: If root.left == NULL AND root.right == NULL
Step 4: Print root.data
Step 5: Call DISPLAY_LEAF(root.left)
Step 6: Call DISPLAY_LEAF(root.right)
Step 7: Stop

4. Find Depth of Tree
Step 1: Start
Step 2: If root == NULL, return 0
Step 3: Compute leftHeight = HEIGHT(root.left)
Step 4: Compute rightHeight = HEIGHT(root.right)

Step 5: Return max(leftHeight, rightHeight) + 1
Step 6: Stop

5. Copy Tree
Step 1: Start
Step 2: If root == NULL, return NULL
Step 3: Create newNode = CREATE_NODE(root.data)
Step 4: newNode.left = COPY(root.left)
Step 5: newNode.right = COPY(root.right)
Step 6: Return newNode
Step 7: Stop

B. Binary Search Tree –
1. BST Insertion
Step 1: Start
Step 2: If root == NULL
Step 3: Return CREATE_NODE(data)
Step 4: If data ≥ root.data
Step 5: Set root.r = INSERT(root.r, data)
Step 6: Else
Step 7: Set root.l = INSERT(root.l, data)
Step 8: Return root
Step 9: Stop

2. BST Display
Step 1: Start
Step 2: If root == NULL, return
Step 3: Call DISPLAY(root.l)
Step 4: Print root.data
Step 5: Call DISPLAY(root.r)
Step 6: Stop

3. BST Delete
a. FindMin Function
Step 1: Start
Step 2: While root != NULL AND root.l != NULL
Step 3: Set root = root.l
Step 4: Return root
Step 5: Stop

b. Delete Function
Step 1: Start
Step 2: If root == NULL, return root
Step 3: If data < root.data
Step 4: Set root.l = DELETE_NODE(root.l, data)
Step 5: Else if data > root.data
Step 6: Set root.r = DELETE_NODE(root.r, data)
Step 7: Else (Node Found)
Case 1: No Left Child

Step 8: If root.l == NULL
Step 9: Store root.r in temp
Step 10: Delete root
Step 11: Return temp
Case 2: No Right Child
Step 12: Else if root.r == NULL
Step 13: Store root.l in temp
Step 14: Delete root
Step 15: Return temp
Case 3: Two Children
Step 16: Else
Step 17: Find minimum node in right subtree
Step 18: Copy its value into current node
Step 19: Set root.r = DELETE_NODE(root.r, copied_value)
Step 20: Return root
Step 21: Stop

4. BST Search
Step 1: Start
Step 2: If root == NULL
Step 3: Print "Value not found"
Step 4: Return
Step 5: If root.data == val follow step 6-7
Step 6: Print "Value found"
Step 7: Return
Step 8: If val < root.data follow step 9
Step 9: Call SEARCH(root.l, val)
Step 10: Else follow step 11
Step 11: Call SEARCH(root.r, val)
Step 12: Stop

5. BST BFS Traversal
Step 1: Start
Step 2: If root == NULL, return
Step 3: Create empty queue q
Step 4: Insert root into queue
Step 5: While queue is not empty
Step 6: Remove front node → temp
Step 7: Print temp.data
Step 8: If temp.l != NULL
Step 9: Insert temp.l into queue
Step 10: If temp.r != NULL
Step 11: Insert temp.r into queue
Step 12: End While
Step 13: Stop

Pseudocodes:
1. Creating a Node

```
Structure Node:
   Data: Value
```

```
    Left: Pointer to Node (NULL)
    Right: Pointer to Node (NULL)

  Function CreateNode(value):
    NewNode = Allocate memory for Node
    NewNode.Data = value
    NewNode.Left = NULL
    NewNode.Right = NULL
    Return NewNode
```

2. Inserting a Node in tree

```
  Function InsertBST(Root, value):
    If Root is NULL:
        Return CreateNode(value)
    If value < Root.Data:
        Root.Left = InsertBST(Root.Left, value)
    Else if value > Root.Data:
        Root.Right = InsertBST(Root.Right, value)
    Return Root
```

3. Searching for a Node (for BST)

```
  Function SearchBST(Root, key):
        If Root is NULL or Root.Data == key:
        Return Root

        If key < Root.Data:
                Return SearchBST(Root.Left, key)
        Else:
                Return SearchBST(Root.Right, key)
```

4. Deleting a Node

```
  Function DeleteBST(Root, key):
        If Root is NULL:
                Return NULL
        If key < Root.Data:
                Root.Left = DeleteBST(Root.Left, key)
        Else if key > Root.Data:
                Root.Right = DeleteBST(Root.Right, key)
        Else: // Found the node to delete
        // Case 1 & 2: One child or No child
                If Root.Left is NULL:
                        Return Root.Right
                If Root.Right is NULL:
                        Return Root.Left
        // Case 3: Two children
                Temp = FindMin(Root.Right) // Find In-order Successor
                Root.Data = Temp.Data
                Root.Right = DeleteBST(Root.Right, Temp.Data)
                Return Root
```

5. Traversals

a. InOrder:

```
Function InOrder(Root):
    If Root is not NULL:
        InOrder(Root.Left)
        Print Root.Data
        InOrder(Root.Right)
```

b. PreOrder:

```
Function PreOrder(Root):
    If Root is not NULL:
        Print Root.Data
        PreOrder(Root.Left)
        PreOrder(Root.Right)
```

c. PostOrder:

```
Function PostOrder(Root):
    If Root is not NULL:
        PreOrder(Root.Left)
        PreOrder(Root.Right)
        Print Root.Data
```

d. LevelOrder: (BFS)

```
Function BFS(Root):
    If Root is NULL:
        Return
    //Create an Empty Queue Q
    Enqueue(Q, Root)

    While Q is not empty:
        Current = Dequeue(Q)
        Print Current.Data
    If Current.Left is not NULL:
        Enqueue(Q, Current.Left)
    If Current.Right is not NULL:
        Enqueue(Q, Current.Right)
```

6. Calculating Depth of Tree

```
Function GetDepth(Root):
    If Root is NULL:
        Return 0
LeftDepth = GetDepth(Root.Left)
RightDepth = GetDepth(Root.Right)
Return Max(LeftDepth, RightDepth) + 1
```

7. Displaying Leaf Nodes

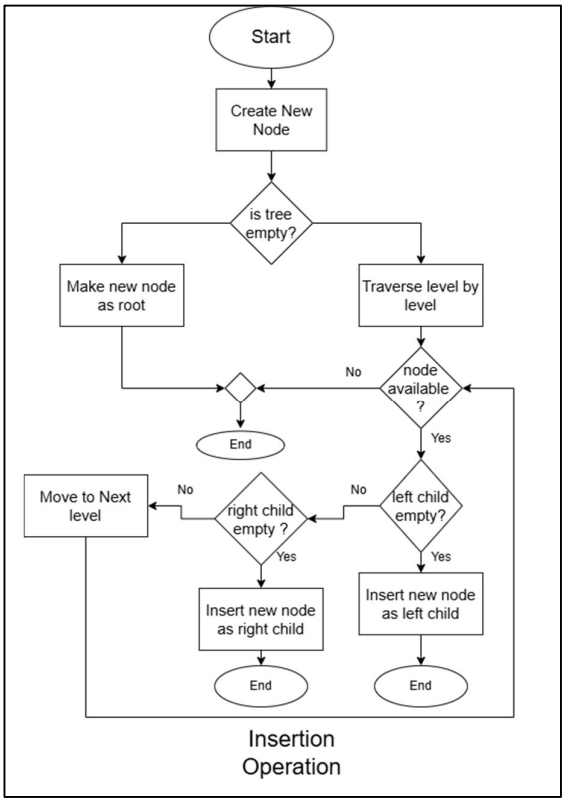
```
Function DisplayLeaf(Root):
    If Root is NULL:
        Return
If Root.Left is NULL and Root.Right is NULL:
    Print Root.Data
DisplayLeaf(Root.Left)
DisplayLeaf(Root.Right)
```

8. Copy Tree

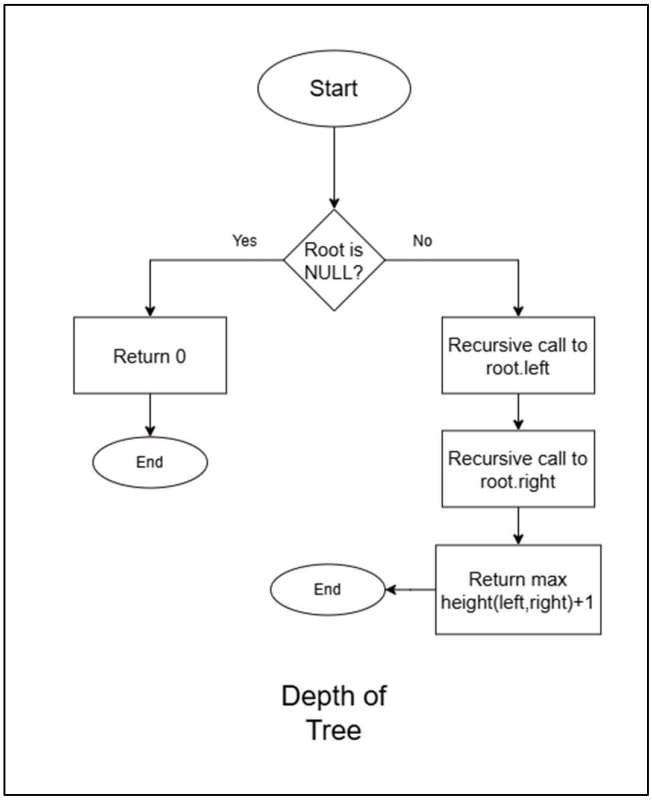
```
Function CopyTree(Root):
        If Root is NULL:
                Return NULL
NewNode = CreateNode(Root.Data)
NewNode.Left = CopyTree(Root.Left)
NewNode.Right = CopyTree(Root.Right)
Return NewNode
```

Flowcharts:

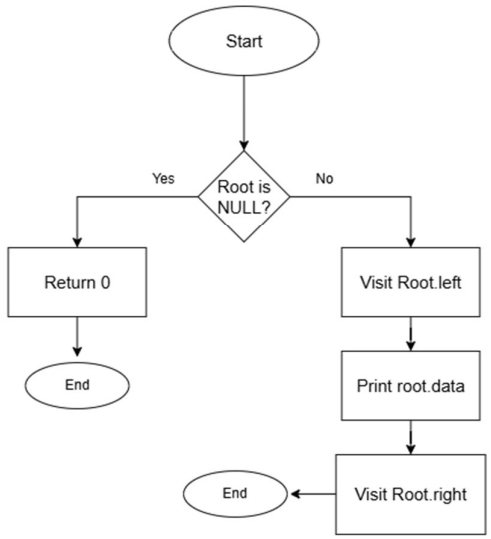
1. Binary Tree Operations



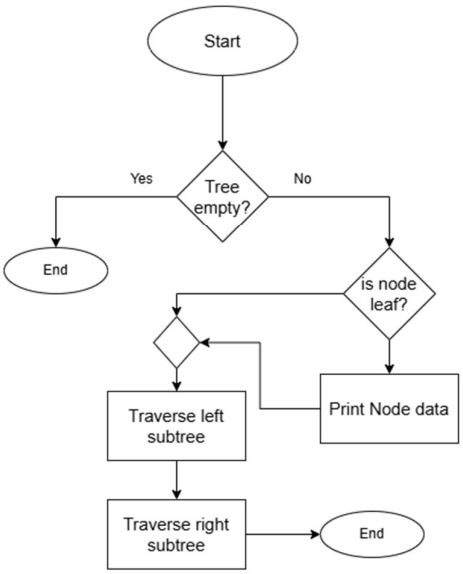
Insertion Operation



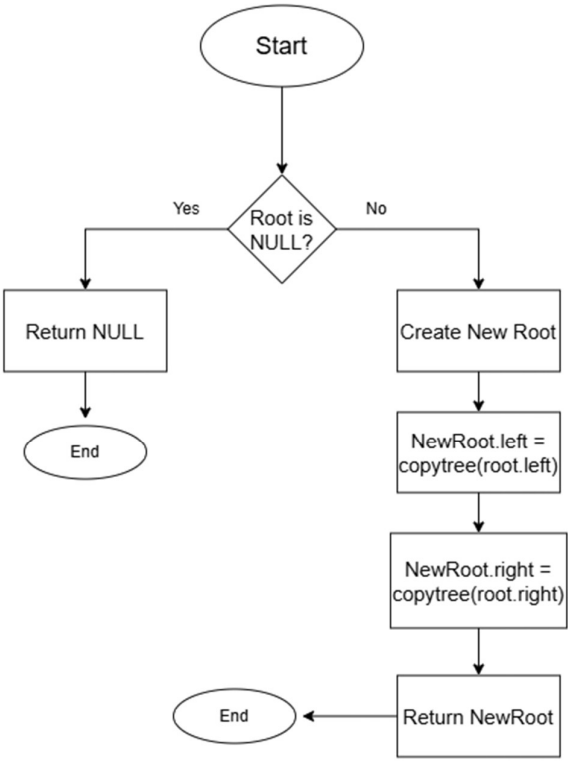
Depth of Tree



Display Tree

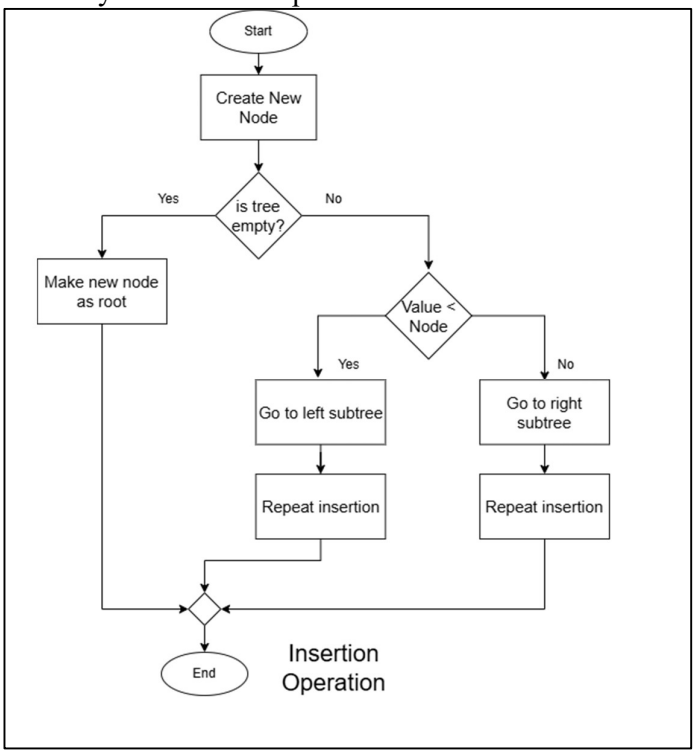


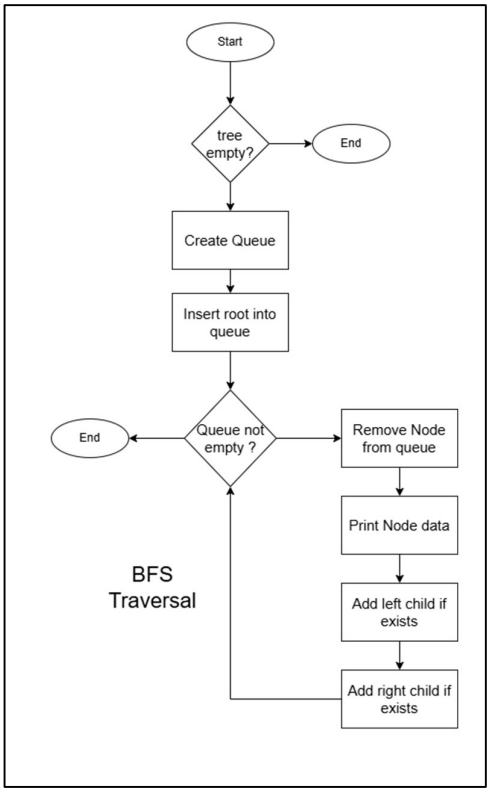
Display leaf nodes of Tree

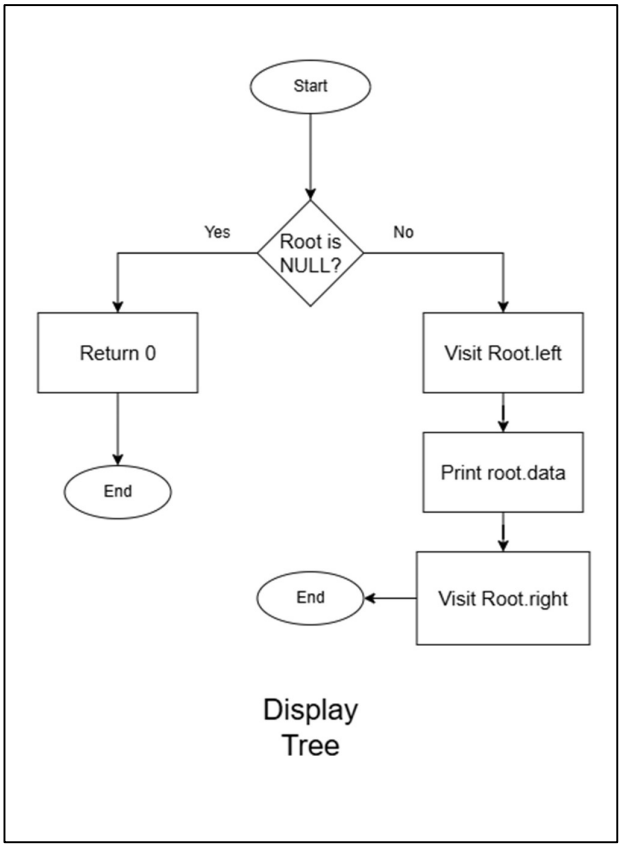


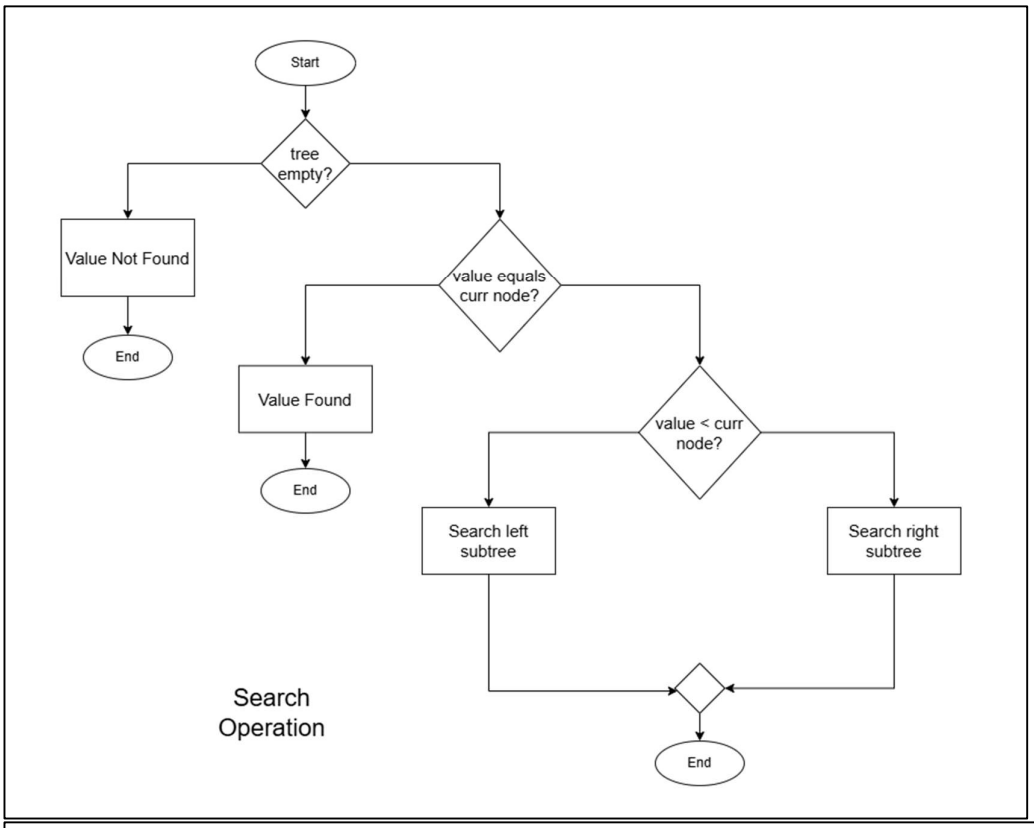
Copy Tree

2. Binary Search Tree Operations

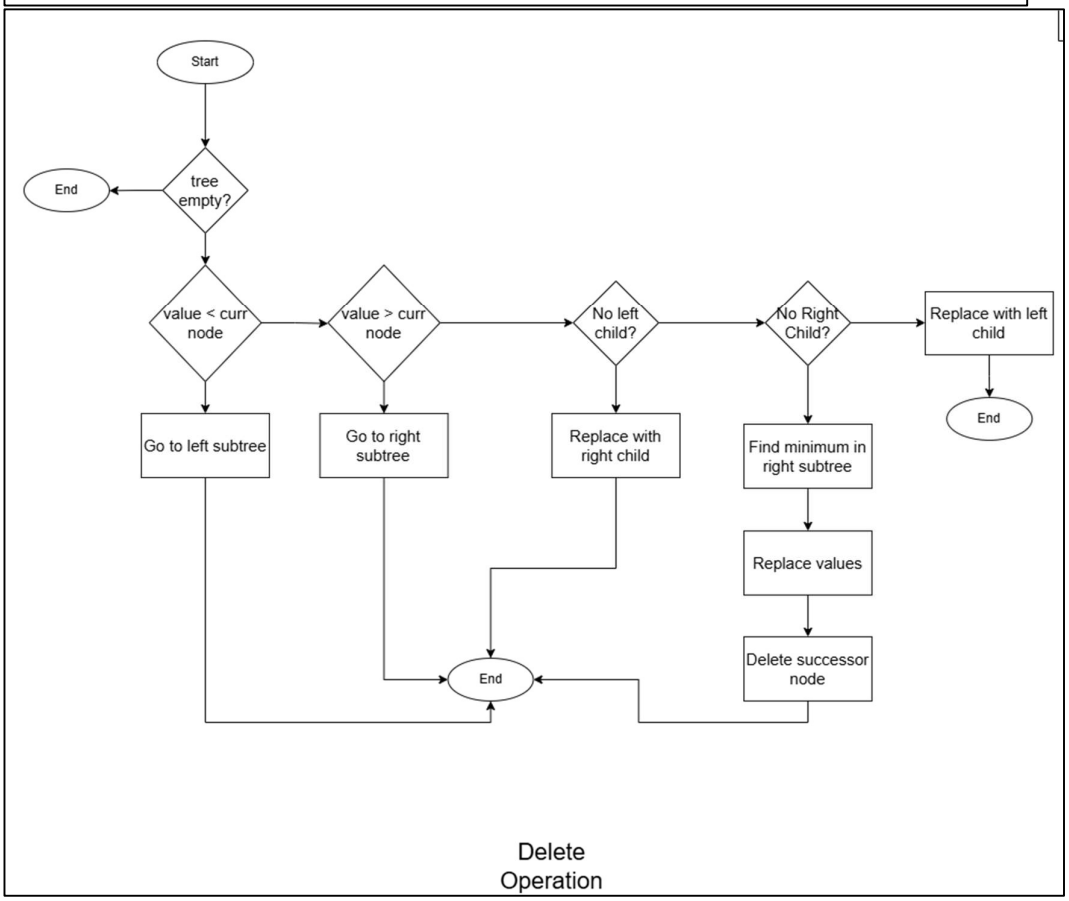








Search Operation



Delete Operation

Codes:

1. BinaryTree.cpp

```
#include <iostream>
using namespace std;

struct Node{
    int data;
    Node* left;
    Node* right;

};

Node* createNode(int data){
    Node* newNode = new Node();
    newNode->data = data;
    newNode->left = nullptr;
    newNode->right = nullptr;
    return newNode;
}

Node* insert(Node* root, int data){
    if(root == nullptr){
        return createNode(data);
    }
    if(data < root->data){
        root->left = insert(root->left,data);
    }else{
        root->right=insert(root->right,data);
    }
    return root;
}

void display(Node* root){
    if(root==nullptr){
        return;
    }
    display(root->left);
    cout<<root->data<<" ";
    display(root->right);
}

int height(Node* root){
    if(root==nullptr){
        return 0;
    }
    int leftH = height(root->left);
    int rightH = height(root->right);
    return max(leftH,rightH)+1;
}

Node* displayLeafNode(Node* root){
    if(root==nullptr){
        return nullptr;
    }
    if(root->left==nullptr && root->right==nullptr){
        cout<<root->data<<" ";
    }
    displayLeafNode(root->left);
    displayLeafNode(root->right);
```

```
    return root;
}

Node* copyTree(Node* root){
    if(root==nullptr){
        return nullptr;
    }
    Node* newNode = createNode(root->data);
    newNode->left = copyTree(root->left);
    newNode->right = copyTree(root->right);
    return newNode;
}

int main(){
    Node* root = nullptr;
    Node* newTree = nullptr;
    root = insert(root,50);
    int choice, data;
    while(true){
        cout<<"Menu:"<<endl;
        cout<<"1. Insert"<<endl;
        cout<<"2. Display - Inorder"<<endl;
        cout<<"3. Height"<<endl;
        cout<<"4. Display Leaf Nodes"<<endl;
        cout<<"5. Copy Tree"<<endl;
        cout<<"6. Exit"<<endl;
        cout<<"Enter your choice: ";
        cin>>choice;
        switch(choice){
            case 1:
                cout<<"\nEnter data to insert : ";
                cin>>data;
                root = insert(root,data);
                break;
            case 2:
                cout<<"\nDisplay Tree: (Inorder) ";
                display(root);
                cout<<endl;
                break;
            case 3:
                cout<<"\nHeight of Tree : "<<height(root)<<endl;
                break;
            case 4:
                cout<<"\nDisplaying Leaf Nodes: ";
                displayLeafNode(root);
                cout<<endl;
                break;
            case 5:
                newTree = copyTree(root);
                cout<<"\nCopied Tree: ";
                display(newTree);
                cout<<endl;
                break;
            case 6:
                cout<<"\nExiting..."<<endl;
                return 0;
            default:
                cout<<"\nInvalid Choice!"<<endl;
                break;
```

```
        }
    }
    return 0;
}
```

2. BinarySearchTree.cpp

```
#include<iostream>
#include<queue>
using namespace std;

struct Node{
    int data;
    Node* l;
    Node* r;
};

Node* create(int data){
    Node* root=new Node();
    root->data=data;
    root->l=NULL;
    root->r=NULL;
    return root;
}

void display(Node* root){
    if(root==NULL){
        return;
    }
    display(root->l);
    cout<<root->data<<" ";
    display(root->r);
}

Node* insert(Node* root, int data){
    if(root==NULL){
        return create(data);
    }
    if(data>=root->data){
        root->r=insert(root->r,data);
    }else{
        root->l=insert(root->l,data);
    }
    return root;
}

Node* minimum(Node* root){
    while(root&&root->l!=NULL){
        root=root->l;
    }
    return root;
}

Node* del(Node* root, int data){
    if(root==NULL){
        return root;
    }
    if(data<root->data){
        root->l=del(root->l,data);
```

```
    }else if(data>root->data){
        root->r=del(root->r,data);
    }else{
        if(root->l==NULL){
            Node* t=root->r;
            delete root;
            return t;
        }else if(root->r==NULL){
            Node* t=root->l;
            delete root;
            return t;
        }else{
            Node* t=minimum(root->r);
            root->data=t->data;
            root->r=del(root->r,t->data);
        }
        return root;
    }
}

int search(Node* root, int val){
    if(root==NULL){
        cout<<"Value not found"<<endl;
        return 0;
    }
    if(root->data==val){
        cout<<"Value found"<<endl;
    }else if(val<root->data){
        search(root->l,val);
    }
    else{
        search(root->r,val);
    }
}

void bfs(Node* root) {
    if(root==NULL) {
        return;
    }
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* temp=q.front();
        q.pop();
        cout<<temp->data<<" ";
        if(temp->l!=NULL){
            q.push(temp->l);
        }
        if (temp->r != NULL){
            q.push(temp->r);
        }
    }
}

int main(){
    Node* root=NULL;
    int ch, val;
    while(true){
    cout<<"Menu:"<<endl;
```

```
    cout<<"1. Insert"<<endl;
    cout<<"2. Display"<<endl;
    cout<<"3. Delete"<<endl;
    cout<<"4. Search"<<endl;
    cout<<"5. BFS"<<endl;
    cout<<"6. Exit"<<endl;
    cout<<"Enter your choice: ";
    cin>>ch;
    switch(ch){
        case 1:
        cout<<"Enter data to insert:";
        cin>>val;
        root=insert(root,val);
        cout<<"Node inserted in tree"<<endl;
        break;
        case 2:
        cout<<"Inorder Tree: ";
        display(root);
        cout<<endl;
        break;
        case 3:
        cout<<"Which value to delete?"<<endl;
        cin>>val;
        root=del(root,val);
        cout<<"Node deleted"<<endl;
        break;
        case 4:
        cout<<"Which value to search?"<<endl;
        cin>>val;
        search(root,val);
        break;
        case 5:
        cout<<"BFS Traversal: "<<endl;
        bfs(root);
        cout<<endl;
        break;
        case 6:
        cout<<"BYE BYE User!"<<endl;
        return 0;
        default:
        cout<<"Invalid Choice!"<<endl;
        break;
    }
    }
    return 0;
}
```

Outputs:

1. Binary Tree

a. Insertion

```
Menu:
1. Insert
2. Display - Inorder
3. Height
4. Display Leaf Nodes
5. Copy Tree
6. Exit
Enter your choice: 1

Enter data to insert : 15
Menu:
1. Insert
2. Display - Inorder
3. Height
4. Display Leaf Nodes
5. Copy Tree
6. Exit
Enter your choice: 1

Enter data to insert : 30
Menu:
1. Insert
2. Display - Inorder
3. Height
4. Display Leaf Nodes
5. Copy Tree
6. Exit
Enter your choice: 1

Enter data to insert : 5
```

```
Menu:
1. Insert
2. Display - Inorder
3. Height
4. Display Leaf Nodes
5. Copy Tree
6. Exit
Enter your choice: 1

Enter data to insert : 24
```

b. Display

```
Menu:
1. Insert
2. Display - Inorder
3. Height
4. Display Leaf Nodes
5. Copy Tree
6. Exit
Enter your choice: 2

Display Tree: (Inorder) 5 15 24 30 50
```

c. Height

```
Enter your choice: 3

Height of Tree : 4
```

d. Leaf Nodes

```
Menu:
1. Insert
2. Display - Inorder
3. Height
4. Display Leaf Nodes
5. Copy Tree
6. Exit
Enter your choice: 4

Displaying Leaf Nodes: 5 24
```

e. Copy Tree

```
Menu:
1. Insert
2. Display - Inorder
3. Height
4. Display Leaf Nodes
5. Copy Tree
6. Exit
Enter your choice: 5

Copied Tree: 5 15 24 30 50
```

2. Binary Search Tree

a. Insertion

```
Menu:
1. Insert
2. Display
3. Delete
4. Search
5. BFS
6. Exit
Enter your choice: 1
Enter data to insert:12
Node inserted in tree
Menu:
1. Insert
2. Display
3. Delete
4. Search
5. BFS
6. Exit
Enter your choice: 1
Enter data to insert:4
Node inserted in tree
Menu:
1. Insert
2. Display
3. Delete
4. Search
5. BFS
6. Exit
Enter your choice: 1
Enter data to insert:6
Node inserted in tree
```

```
Menu:
1. Insert
2. Display
3. Delete
4. Search
5. BFS
6. Exit
Enter your choice: 1
Enter data to insert:8
Node inserted in tree
Menu:
1. Insert
2. Display
3. Delete
4. Search
5. BFS
6. Exit
Enter your choice: 1
Enter data to insert:14
Node inserted in tree
Menu:
1. Insert
2. Display
3. Delete
4. Search
5. BFS
6. Exit
Enter your choice: 1
Enter data to insert:19
Node inserted in tree
```

b. Display

```
Enter your choice: 2
Inorder Tree: 4 6 8 12 14 19
```

c. Delete

```
Enter your choice: 3
Which value to delete?
8
Node deleted
```

d. Search

```
Enter your choice: 4
Which value to search?
19
Value found
```

e. BFS

```
Enter your choice: 5
BFS Traversal:
12 4 14 6 19
```